广汽新能源 前端研发工程师

二面

1.使用 CSS3 实现一个渐变色的按钮，并给出其实现过程；

2.手写一个 Promise，并解释其实现原理；

3.实现一个虚拟滚动列表（Virtual Scroll），能够在滚动大量数据时实现快速展示，并在不需要展示的数据上释放内存；

4.实现一个无限极的树形菜单，并能够支持展开、折叠、搜索、拖拽等功能；

5.使用原生 CSS 实现一个 3D 立体轮播图，可自动播放，可左右切换，可点击切换，且切换过程平滑；

6.实现一个自适应布局组件，并支持任意数量列、自定义列宽、动态添加、删除、排序等操作；

7.实现一个图片懒加载功能，当图片进入可视区域时才进行加载；

8.手写实现一个事件总线（Event Bus），能够在不同组件之间传递事件，并支持同一事件绑定多个回调函数；

9.实现一个拖拽功能，能够在元素内部、元素之间进行拖拽，并支持限制拖拽范围、指定拖拽方向等功能；

10.实现一个 Canvas 绘图组件，并能够支持实时绘制、撤销、重做、导出、导入等功能；

参考答案：

1. 使用 CSS3 实现一个渐变色的按钮

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Gradient Button</title>
    <style>
        .gradient-button {
            background: linear-gradient(to right, #ff7e5f, #feb47b);
```

```
            border: none;
            color: white;
            padding: 15px 30px;
            text-align: center;
            text-decoration: none;
            display: inline-block;
            font-size: 16px;
            margin: 4px 2px;
            cursor: pointer;
            transition: background 0.3s ease;
        }
        .gradient-button:hover {
            background: linear-gradient(to right, #feb47b, #ff7e5f);
        }
    </style>
</head>
<body>
    <button class="gradient-button">Gradient Button</button>
</body>
</html
```

2. 手写一个 Promise，并解释其实现原理

```

class MyPromise {
    constructor(executor) {
        this.state = 'pending';
        this.value = undefined;
        this.reason = undefined;
        this.onFulfilledCallbacks = [];
        this.onRejectedCallbacks = [];

        const resolve = (value) => {
            if (this.state === 'pending') {
                this.state = 'fulfilled';
                this.value = value;
                this.onFulfilledCallbacks.forEach(fn => fn());
            }
        };

        const reject = (reason) => {
            if (this.state === 'pending') {
                this.state = 'rejected';
                this.reason = reason;
                this.onRejectedCallbacks.forEach(fn => fn());
```

```
            }
        };

        try {
            executor(resolve, reject);
        } catch (err) {
            reject(err);
        }
    }

    then(onFulfilled, onRejected) {
        onFulfilled = typeof onFulfilled === 'function' ? onFulfilled : value => value;
        onRejected = typeof onRejected === 'function' ? onRejected : reason => { throw reason };

        return new MyPromise((resolve, reject) => {
            if (this.state === 'fulfilled') {
                setTimeout(() => {
                    try {
                        const x = onFulfilled(this.value);
                        resolve(x);
                    } catch (err) {
                        reject(err);
                    }
                }, 0);
            } else if (this.state === 'rejected') {
                setTimeout(() => {
                    try {
                        const x = onRejected(this.reason);
                        resolve(x);
                    } catch (err) {
                        reject(err);
                    }
                }, 0);
            } else if (this.state === 'pending') {
                this.onFulfilledCallbacks.push(() => {
                    setTimeout(() => {
                        try {
                            const x = onFulfilled(this.value);
                            resolve(x);
                        } catch (err) {
                            reject(err);
                        }
                    }, 0);
                });
                this.onRejectedCallbacks.push(() => {
```

```
                    setTimeout(() => {
                        try {
                            const x = onRejected(this.reason);
                            resolve(x);
                        } catch (err) {
                            reject(err);
                        }
                    }, 0);
                });
            }
        });
    }
}

// 使用示例
const promise = new MyPromise((resolve, reject) => {
    setTimeout(() => {
        resolve('Success!');
    }, 1000);
});

promise.then(value => {
    console.log(value); // Success!
})
```

3. 实现一个虚拟滚动列表（Virtual Scroll）

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Virtual Scroll</title>
    <style>
        #container {
            height: 200px;
            overflow: auto;
            border: 1px solid #ccc;
        }
        #list {
            position: relative;
            will-change: transform;
        }
        .item {
```

```
            height: 50px;
            border-bottom: 1px solid #eee;
            box-sizing: border-box;
            padding: 10px;
        }
    </style>
</head>
<body>
    <div id="container">
        <div id="list"></div>
    </div>

    <script>
        const container = document.getElementById('container');
        const list = document.getElementById('list');
        const itemHeight = 50;
        const totalItems = 10000;

        const visibleCount = Math.ceil(container.clientHeight / itemHeight);
        const buffer = 5;
        const renderCount = visibleCount + buffer;

        const render = (start) => {
            const fragment = document.createDocumentFragment();
            for (let i = start; i < start + renderCount && i < totalItems;
i++) {
                const item = document.createElement('div');
                item.className = 'item';
                item.style.transform = translateY(${i * itemHeight}px);
                item.textContent = Item ${i + 1};
                fragment.appendChild(item);
            }
            list.innerHTML = '';
            list.appendChild(fragment);
        };

        const onScroll = () => {
            const scrollTop = container.scrollTop;
            const start = Math.floor(scrollTop / itemHeight);
            render(start);
        };

        list.style.height = ${totalItems * itemHeight}px;
        container.addEventListener('scroll', onScroll);

        render(0);
    </script>
```

```
</body>
</html
```

4. 实现一个无限极的树形菜单

由于实现无限极的树形菜单较为复杂，下面仅展示基本的结构和核心功能：

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Tree Menu</title>
    <style>
        .tree ul {
            list-style: none;
            padding-left: 20px;
        }
        .tree li {
            margin: 5px 0;
        }
        .tree li label {
            cursor: pointer;
        }
    </style>
</head>
<body>
    <div class="tree">
        <ul id="tree-root"></ul>
    </div>

    <script>
        const data = [
            {
                text: 'Item 1',
                children: [
                    { text: 'Item 1.1', children: [{ text: 'Item 1.1.1' }] },
                    { text: 'Item 1.2' }
                ]
            },
            {
                text: 'Item 2',
                children: [
                    { text: 'Item 2.1' },
                    { text: 'Item 2.2', children: [{ text: 'Item 2.2.1' }] }
```

```
                ]
            }
        ];

        const createTree = (data) => {
            const ul = document.createElement('ul');
            data.forEach(item => {
                const li = document.createElement('li');
                const label = document.createElement('label');
                label.textContent = item.text;
                li.appendChild(label);
                if (item.children) {
                    li.appendChild(createTree(item.children));
                }
                ul.appendChild(li);
            });
            return ul;
        };

        const root = document.getElementById('tree-root');
        root.appendChild(createTree(data));
    </script>
</body>
</html
```

5. 使用原生 CSS 实现一个 3D 立体轮播图

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>3D Carousel</title>
    <style>
        .carousel {
            perspective: 1000px;
        }
        .carousel-inner {
            position: relative;
            width: 100%;
            height: 300px;
            transform-style: preserve-3d;
            animation: rotate 10s infinite linear;
        }
```

```
        .carousel-item {
            position: absolute;
            width: 100%;
            height: 100%;
            background: #ccc;
            line-height: 300px;
            text-align: center;
            font-size: 2em;
            backface-visibility: hidden;
        }
        .carousel-item:nth-child(1) { transform: rotateY(0deg) translateZ(400px); }
        .carousel-item:nth-child(2) { transform: rotateY(120deg) translateZ(400px); }
        .carousel-item:nth-child(3) { transform: rotateY(240deg) translateZ(400px); }

        @keyframes rotate {
            from { transform: rotateY(0deg); }
            to { transform: rotateY(360deg); }
        }
    </style>
</head>
<body>
    <div class="carousel">
        <div class="carousel-inner">
            <div class="carousel-item">1</div>
            <div class="carousel-item">2</div>
            <div class="carousel-item">3</div>
        </div>
    </div>
</body>
</html
```

6. 实现一个自适应布局组件

实现一个自适应布局组件

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Responsive Layout</title>
    <style>
```

```
        .container {
            display: flex;
            flex-wrap: wrap;
        }
        .column {
            flex: 1;
            min-width: 200px;
            margin: 10px;
            background: #f0f0f0;
            padding: 20px;
            box-sizing: border-box;
        }
    </style>
</head>
<body>
    <div class="container">
        <div class="column">Column 1</div>
        <div class="column">Column 2</div>
        <div class="column">Column 3</div>
    </div>

    <script>
        // 动态添加列
        const container = document.querySelector('.container');
        const addColumn = (text) => {
            const column = document.createElement('div');
            column.className = 'column';
            column.textContent = text;
            container.appendChild(column);
        };

        addColumn('Column 4');
    </script>
</body>
</html
```

7. 实现一个图片懒加载功能

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Lazy Load Images</title>
```

```
    <style>
        .image-container {
            height: 1000px;
        }
        .lazy {
            width: 100%;
            height: 300px;
            display: block;
            background: #f0f0f0;
        }
    </style>
</head>
<body>
    <div class="image-container">
        < img class="lazy" data-src="https://via.placeholder.com/600x300" alt="Lazy Image">
    </div>

    <script>
        document.addEventListener('DOMContentLoaded', () => {
            const lazyImages = document.querySelectorAll('.lazy');

            const lazyLoad = (entries, observer) => {
                entries.forEach(entry => {
                    if (entry.isIntersecting) {
                        const img = entry.target;
                        img.src = img.dataset.src;
                        img.classList.remove('lazy');
                        observer.unobserve(img);
                    }
                });
            };

            const observer = new IntersectionObserver(lazyLoad);
            lazyImages.forEach(image => observer.observe(image));
        });
    </script>
</body>
</html
```

8. 手写实现一个事件总线（Event Bus）

```

class EventBus {
    constructor() {
```

```
        this.events = {};
    }

    on(event, listener) {
        if (!this.events[event]) {
            this.events[event] = [];
        }
        this.events[event].push(listener);
    }

    off(event, listenerToRemove) {
        if (!this.events[event]) return;
        this.events[event] = this.events[event].filter(listener => listener
!== listenerToRemove);
    }

    emit(event, data) {
        if (!this.events[event]) return;
        this.events[event].forEach(listener => listener(data));
    }
}

// 使用示例
const bus = new EventBus();

const listener = (data) => console.log('Event received:', data);
bus.on('test', listener);
bus.emit('test', { a: 1 });
bus.off('test', listener)
```

9. 实现一个拖拽功能

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Drag and Drop</title>
    <style>
        .draggable {
            width: 100px;
            height: 100px;
            background: #f0f0f0;
            position: absolute;
```

```
            cursor: move;
        }
    </style>
</head>
<body>
    <div class="draggable" id="draggable">Drag me</div>

    <script>
        const draggable = document.getElementById('draggable');

        draggable.addEventListener('mousedown', (e) => {
            let offsetX = e.clientX - parseInt(window.getComputedStyle(draggable).left);
            let offsetY = e.clientY - parseInt(window.getComputedStyle(draggable).top);

            const onMouseMove = (e) => {
                draggable.style.left = `${e.clientX - offsetX}px`;
                draggable.style.top = `${e.clientY - offsetY}px`;
            };

            const onMouseUp = () => {
                document.removeEventListener('mousemove', onMouseMove);
                document.removeEventListener('mouseup', onMouseUp);
            };

            document.addEventListener('mousemove', onMouseMove);
            document.addEventListener('mouseup', onMouseUp);
        });
    </script>
</body>
</html
```

10. 实现一个 Canvas 绘图组件

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Canvas Drawing</title>
    <style>
        canvas {
            border: 1px solid #ccc;
```

```
        }
    </style>
</head>
<body>
    <canvas id="canvas" width="800" height="600"></canvas>
    <button id="undo">Undo</button>
    <button id="redo">Redo</button>

    <script>
        const canvas = document.getElementById('canvas');
        const ctx = canvas.getContext('2d');
        let drawing = false;
        let history = [];
        let redoStack = [];

        const saveState = () => {
            history.push(canvas.toDataURL());
            if (history.length > 10) history.shift();
        };

        const restoreState = (state) => {
            const img = new Image();
            img.src = state;
            img.onload = () => ctx.drawImage(img, 0, 0);
        };

        canvas.addEventListener('mousedown', () => {
            drawing = true;
            saveState();
            redoStack = [];
        });

        canvas.addEventListener('mouseup', () => {
            drawing = false;
        });

        canvas.addEventListener('mousemove', (e) => {
            if (!drawing) return;
            const rect = canvas.getBoundingClientRect();
            const x = e.clientX - rect.left;
            const y = e.clientY - rect.top;
            ctx.lineTo(x, y);
            ctx.stroke();
        });

        document.getElementById('undo').addEventListener('click', () => {
            if (history.length === 0) return;
```

```
            redoStack.push(history.pop());
            ctx.clearRect(0, 0, canvas.width, canvas.height);
            if (history.length > 0) restoreState(history[history.length - 1]);
        });

        document.getElementById('redo').addEventListener('click', () => {
            if (redoStack.length === 0) return;
            const state = redoStack.pop();
            restoreState(state);
            history.push(state);
        });
    </script>
</body>
</html>
```